

BookStore

Full Stack MERN Application — Project Documentation

1. Introduction

- **Project Title:** BookStore
- **Developer:** Nimmala Bhuvana Lakshmi
- **Technology Stack:** MongoDB, Express.js, React.js, Node.js (MERN)

2. Project Overview

Purpose

BookStore is a full-stack online book shopping platform that connects readers, independent sellers, and platform administrators in a single marketplace. Users can browse, search, and purchase books from multiple sellers; sellers can list and manage their own inventory and fulfil orders; and administrators oversee the entire platform, ensuring quality control across users, sellers, books, and orders. The application is fully responsive across mobile, tablet, and desktop devices.

Key Features

Users

- Register and log in securely
- Browse, search, and filter books by genre, author, price, and rating
- Add books to cart and complete checkout
- View order history and track order status
- Leave reviews and ratings on purchased books
- Update personal profile information

Sellers

- Register and log in with a dedicated seller account
- Add, edit, and delete book listings with cover image uploads
- View and fulfil incoming orders for their books
- Manage business profile information

Admins

- Manage all registered users (view, update, remove)
- Approve and manage seller accounts
- Oversee and moderate all book listings
- Monitor and manage all orders placed on the platform
- Maintain full administrative control over the system

3. Architecture

- **Frontend:** Built with React (Vite) using functional components and React Router DOM for client-side navigation. Role-based protected routes separate the User, Seller, and Admin experiences. Axios handles all communication with the backend REST API, and the interface is styled with Bootstrap for a clean, responsive layout.
- **Backend:** Built with Node.js and Express.js, following the MVC (Model-View-Controller) architectural pattern. Routes map incoming requests to controllers, controllers contain the business logic, and Mongoose models define the data schema and handle interaction with MongoDB. Middleware handles authentication, role-based authorization, and file uploads.
- **Database:** MongoDB is used as the primary data store, with Mongoose as the ODM. Core collections include Users, Sellers, Admins, Books, Orders, and Reviews. Relationships (such as User–Order and Book–Seller) are modelled using MongoDB references (ObjectId) between collections.

4. Setup Instructions

Prerequisites

- Node.js (v18 or later)
- MongoDB (local installation or a MongoDB Atlas cluster)
- npm (Node Package Manager)
- Git

Installation

1. Clone the repository:

```
git clone <repository-url>
cd bookstore-mern
```

2. Install backend dependencies:

```
cd server
npm install
```

3. Install frontend dependencies:

```
cd ../client
npm install
```

4. Configure environment variables:

Copy server/.env.example to server/.env and provide the following values:

```
MONGODB_URI=<your-mongodb-connection-string>
JWT_SECRET=<your-jwt-secret-key>
PORT=5000
```

5. Folder Structure

- **Client:** The React frontend is organised by role for clarity and separation of concerns.

```
client/src/
├─ Admin/      → Ahome, Alogin, Anavbar, Asignup, Users, Seller, Items
├─ Seller/     → Addbook, Book, MyProducts, Orders, Shome, Slogin, Ssignup
├─ User/       → Login, Signup, Uhome, Products, Uitem, MyOrders, OrderItem
└─ Components/ → Home, Footer, Navbar, Loader, PrivateRoute
```

- **Server:** The Node.js/Express backend follows the MVC pattern.

```
server/
├─ config/connect.js      → MongoDB connection setup
├─ controllers/           → AdminControllers, SellerControllers, UsersController
├─ middlewares/           → authMiddleware, upload (Multer)
├─ models/                → Admin, Seller, Users schemas
├─ routes/                → adminRoutes, sellerRoutes, userRoutes
├─ uploads/               → Stored book cover images
├─ .env.example
└─ server.js              → Application entry point
```

6. Running the Application

Once dependencies are installed and environment variables are configured, start the backend and frontend servers in separate terminals:

Backend:

```
cd server
npm run dev      (or: node server.js)
```

Frontend:

```
cd client
npm run dev
```

The backend runs on <http://localhost:5000> and the frontend development server on <http://localhost:5173> (default Vite port).

7. API Documentation

The table below summarises the key API endpoints exposed by the backend. All protected routes require a valid JWT sent in the Authorization header, and role-based access is enforced by `authMiddleware.js`.

Method	Endpoint	Access	Description
POST	/api/users/register	Public	Register a new user account
POST	/api/users/login	Public	Authenticate user and return JWT
GET	/api/books	Public	Retrieve all books with search/filter support
GET	/api/books/:id	Public	Retrieve details of a single book
POST	/api/orders	User	Place a new order (creates order, updates stock)
GET	/api/orders/my-orders	User	Get logged-in user's order history
POST	/api/seller/register	Public	Register a new seller account
POST	/api/seller/books	Seller	Add a new book listing with image upload

PUT	/api/seller/books/:id	Seller	Edit an existing book listing
DELETE	/api/seller/books/:id	Seller	Remove a book listing
GET	/api/seller/orders	Seller	View orders placed for seller's books
GET	/api/admin/users	Admin	View and manage all registered users
GET	/api/admin/sellers	Admin	Approve and manage seller accounts
GET	/api/admin/orders	Admin	View and oversee all platform orders

8. Authentication

BookStore uses JSON Web Tokens (JWT) for stateless authentication and bcryptjs for secure password hashing.

- Passwords are hashed using bcryptjs before being stored in the database — plain-text passwords are never saved.
- On successful login, the server issues a signed JWT containing the user's ID and role.
- The token is sent by the client in the Authorization header (Bearer scheme) with each protected request.
- authMiddleware.js verifies the token's validity and checks the user's role before granting access to role-specific routes (User, Seller, or Admin).
- Tokens have a defined expiry, after which the user must log in again.

9. User Interface

The user interface is built with React and styled using Bootstrap for a clean, consistent, and fully responsive experience across mobile, tablet, and desktop screens. Each role (User, Seller, Admin) has a dedicated navbar and dashboard tailored to its permissions.

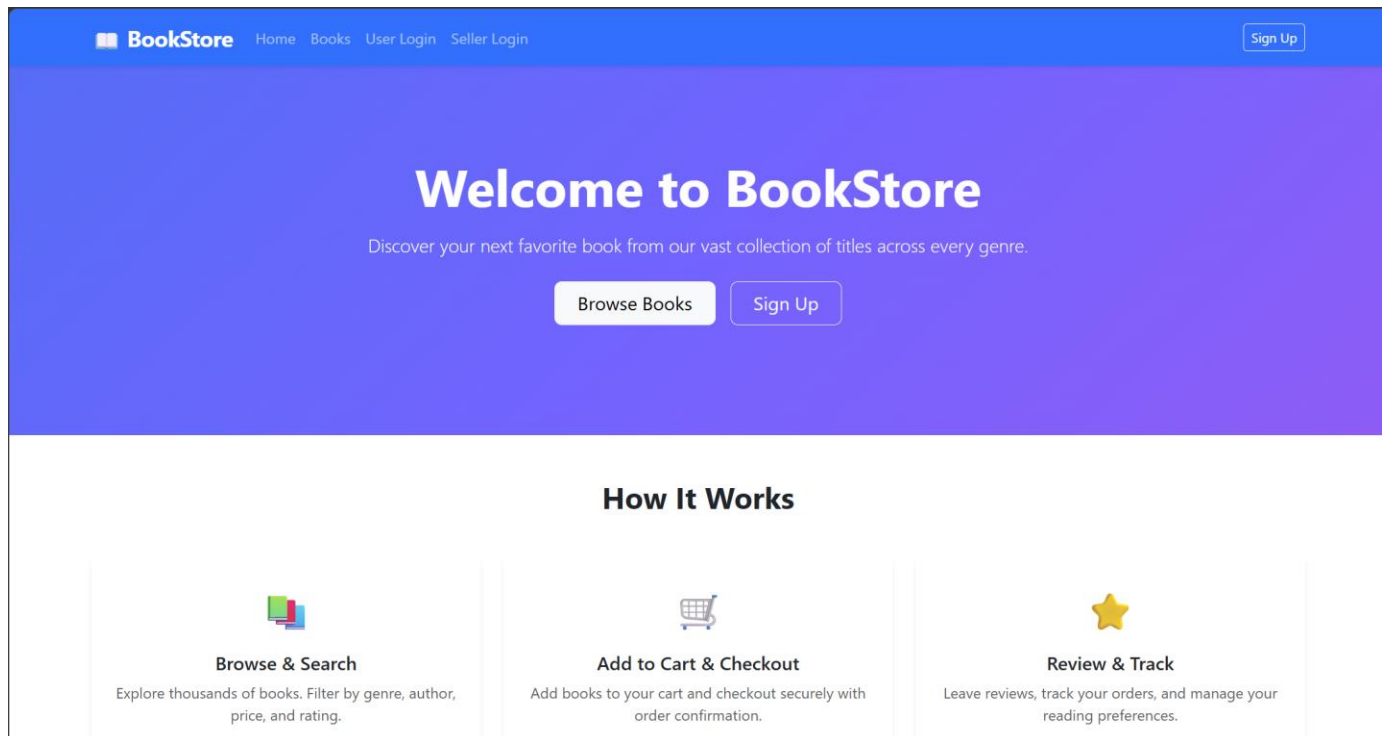
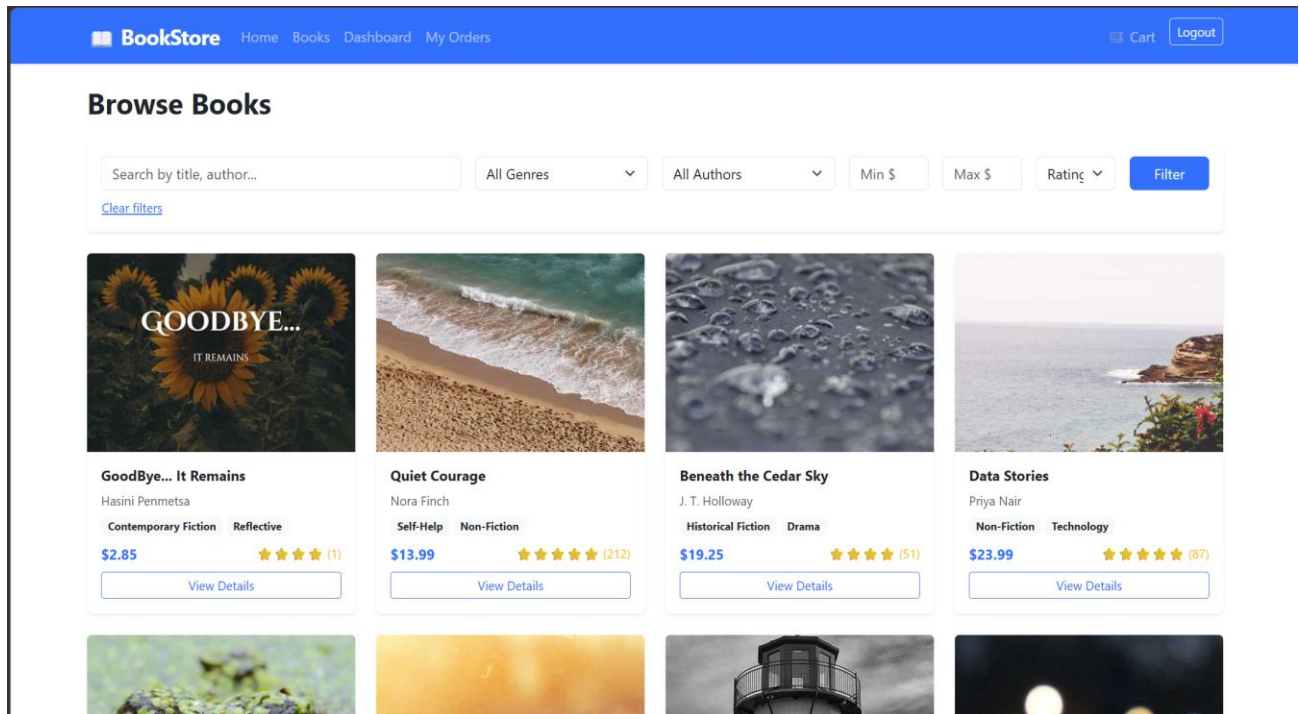
- Book cards with cover images, price, and rating for easy browsing
- Search and filter bar for genre, author, price range, and rating
- Cart and checkout flow with order confirmation
- Seller dashboard for managing listings and orders
- Admin dashboard for platform-wide oversight
- Success and error alerts/toasts for clear user feedback

10. Testing

As a solo-developer project, testing was carried out primarily through structured manual testing across all three roles.

- Manually tested registration, login, and logout flows for Users, Sellers, and Admins
- Verified role-based route protection by attempting unauthorized access to restricted pages
- Tested book creation, editing, deletion, and image upload from the Seller dashboard
- Validated cart, checkout, and order placement, including stock updates
- Cross-checked API responses using Postman for all major endpoints
- Tested UI responsiveness across mobile, tablet, and desktop viewports

11. Screenshots or Demo



12. Known Issues

- Pagination not yet implemented for large book catalogues.

- **Search Limitations:** The current search functionality relies on simple queries, which may not support advanced fuzzy searching or complex natural language inputs.

13. Future Enhancements

- Integration of a real payment gateway (Razorpay / Stripe) for online checkout
- Advanced analytics dashboard for Admins and Sellers (sales trends, top-selling books, revenue reports)
- Wishlist functionality for users
- Email notifications for order confirmation and status updates
- Recommendation engine based on browsing and purchase history
- Automated testing suite (unit and integration tests) to complement manual testing
- Pagination and infinite scroll for large book catalogues